



DÜZCE
ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

2025-2026 AKADEMİK YILI
YAZ DÖNEMİ

BM498 BİLGİSAYAR MÜHENDİSLİĞİ MEZUNİYET TEZİ

Ders Sorumlusu:
Prof. Dr. Yusuf ALTUN

KONTROL AKIŞI DÜZLEŞTİRME
OBFUSCATION'ININ TERSİNE
MÜHENDİSLİĞİ

Hazırlayan:
Senanur ÖZBAĞ
Öğrenci No:
3688951466

BM498 MEZUNİYET TEZİ DEĞERLENDİRME VE SÖZLÜ SINAV TUTANAĞI		
ÖĞRENCİLER (NUMARA/AD-SOYAD): 36889514666/Senanur Özbağ		
DANIŞMAN: Prof. Dr. Yusuf Altun		
Değerlendirme Kriteri	Puan Aralığı	Puan
Yazılı Çalışma Biçimsel Değerlendirme (10 Puan)		
Çalışma kılavuza uygun olarak hazırlanmış mı?	0-10	
Problemin tanımı açıkça yapılmış mı?	0-5	
Literatür taraması yeterli, güncel ve sistematik bir yaklaşımla (PRISMA vb.) gerçekleştirilmiş mi?	0-5	
Geliştirilecek yazılımın/donanımın mimarisini içeren blok şeması (yazılımlar için veri akış şeması da olabilir) çizilerek açıklanmış mı?	0-5	
Kullanılan materyal ve metot yeterli düzeyde anlatılmış mı?	0-5	
Projede kullanılan veri setleri FAIR (Findable, Accessible, Interoperable, Reusable) ilkelerine uygun şekilde yönetilmiş mi?	0-5	
Tasarımın uygulamasında ortaya çıkan uyumsuzluklar ve aksaklıklar belirtilerek çözüm yöntemleri tartışılmış mı?	0-5	
Yapılan işlerin zorluk derecesi?	0-5	
Yapılan sunum başarılı mı?	0-8	
Soruları yanıtlama yetkinliği?	0-7	
Öğrenci dönem içerisindeki raporlarını düzenli olarak hazırladı mı?	0-10	
Etik ve Mesleki Sorumluluk (15 Puan)		
Proje etik ilkelere uygun mu? (İntihal, etik beyan, veri ve yapay zekâ kullanımı, insan verisi kullanımı – Helsinki Deklarasyonu vb.). ☐ İntihal beyanı mevcut ☐ ISO 690 uygun kaynak gösterimi yapılmış ☐ İnsan verisi kullanılmışsa etik izin / onam mevcut ☐ Helsinki Deklarasyonu ilkelerine uygun	0-5	
Projede mühendislik etiği ve mesleki sorumluluk bilinciyle hareket edilmiş mi?	0-5	
Mühendislik uygulamalarında ulusal/uluslararası standartlar belirtilmiş (ISO 9001, ISO 17025, IEEE) ve uygulanmış mı?	0-5	
Proje sürecinde proje yönetimi, risk yönetimi ve değişiklik yönetimi yaklaşımları uygulanmış mı? (İş planı, görev dağılımı, risk analizi)	0-4	
Girişimcilik ve yenilikçilik boyutu değerlendirilmiş mi? (Özgün çözüm, katma değer, ticari potansiyel)	0-4	
Sürdürülebilir kalkınma perspektifi (çevresel, ekonomik ve sosyal sürdürülebilirlik) gözetilmiş mi?	0-4	
Sürdürülebilir Yazılım Geliştirmede Yeniden Üretilebilirlik (Reproducibility) ilkesi uygulanmış mı?	0-3	
Toplam	0-100	

BEYAN

Bu tez çalışmasının kendi çalışmam olduğunu, tezin planlanmasından yazımına kadar bütün aşamalarda etik dışı davranışımın olmadığını, bu tezdeki bütün bilgileri akademik ve etik kurallar içinde elde ettiğimi, bu tez çalışmasıyla elde edilmeyen bütün bilgi ve yorumlara kaynak gösterdiğimi ve bu kaynakları da kaynaklar listesine aldığımı, yine bu tezin çalışılması ve yazımı sırasında patent ve telif haklarını ihlal edici bir davranışımın olmadığını beyan ederim.

19 Haziran 2026

Senanur ÖZBAĞ

ÜRETKEN YAPAY ZEKA KULLANIM BEYANI

19 Haziran 2026

Bu tez çalışmasını hazırlarken Claude, ChatGPT üretken yapay zekâ programlarından destek aldığımı beyan ederim. Tezimin hazırlığı aşamasında üretken yapay zekâ programlarından dil çevirisi, bilimsel makaleye erişim desteği aldım. Üretken yapay zekâ programlarından aldığım bilgilerin doğruluğunu kontrol ettiğimi bildiririm.

Herhangi bir zamanda, çalışmamla ilgili yaptığım bu beyana aykırı bir durumun saptanması durumunda, ortaya çıkacak tüm ahlaki ve hukuki sonuçları kabul ettiğimi bildiririm.

Senanur ÖZBAĞ

TEŐEKKÖR

Lisans öğrenimimde ve bu tezin hazırlanmasında gösterdiği her türlü destek ve yardımdan dolayı çok değerli hocam Prof. Dr. Yusuf ALTUN'a en içten dileklerle teşekkür ederim.

Tez çalışmam boyunca değerli katkılarını esirgemeyen eş danışmanım Prof. Dr. Kemal AKYOL'a da şükranlarımı sunarım.

Bu çalışma boyunca yardımlarını ve desteklerini esirgemeyen sevgili aileme ve çalışma arkadaşlarıma sonsuz teşekkürlerimi sunarım.

19 Haziran 2026

Adı Soyadı

İÇİNDEKİLER

İçindekiler

ÖZET	vii
ABSTRACT	viii
1. GİRİŞ	1
1.1. LİTERATÜR ÖZETİ	1
1.2. TEZİN AMACI	3
1.3. LİTERATÜRE KATKI	4
2. MATERYAL VE YÖNTEM	5
2.1. METERYAL	5
2.2. YÖNTEM	6
3. BÖLÜM	8
3.1. BULGULAR VE TARTIŞMA	8
3.1.1. DENEYSEL KURULUM	8
3.2. DİSPATCHER TESPİTİ SONUÇLARI	9
3.3. SEMBOLİK YOL KEŞFİ VE CFG YENİDEN İNŞASI	11
3.4. PERFORMANS VE ÖLÇEKLENEBİLİRLİK	12
3.5. MEVCUT YÖNTEMLERLE KARŞILAŞTIRMA	12
3.6. TARTIŞMA	13
4. BÖLÜM sonuçlar ve öneriler	14
5. Kaynaklar	16
6. EKLER	18
ÖZGEÇMİŞ	24

ÖZET

KONTROL AKIŞI DÜZLEŞTİRME VE OBFUSCATION'ININ TERSİNE MÜHENDİSLİĞİ

Senanur ÖZBAĞ

Düzce Üniversitesi

Mühendislik Fakültesi Bilgisayar Mühendisliği Bitirme Tezi

Danışman: Prof. Dr. Yusuf ALTUN

Ağustos 2026, 24 sayfa

Kontrol akışı düzleştirme (Control Flow Flattening, CFF), bir programın iç içe geçmiş blok yapısını tek bir merkezi dispatcher ve durum değişkeni etrafında yeniden düzenleyerek statik ve dinamik analizi zorlaştıran, yaygın kullanılan bir yazılım gizleme (obfuscation) tekniğidir. Bu tez, CFF ile korunmuş Windows PE (x86/x64) ikili dosyalarının tersine mühendisliği için, statik ön işleme ile dinamik/sembolik yürütmeyi birleştiren beş aşamalı bir metodoloji önermektedir. Çalışma kapsamında önce alanın literatürü sistematik olarak taranmış, tekniğin kökeninden güncel deobfuscation yaklaşımlarına kadar temel açıklar tespit edilmiş; ardından IDA Pro/Ghidra, Triton ve angr araçlarını temel alan bir analiz hattı tasarlanmıştır. Yöntemin çekirdek varsayımını sınamak amacıyla, elle CFF uygulanmış bir pilot program üzerinde gerçek bir deney yürütülmüştür: angr'ın statik CFG analizi, dispatcher yapısını grafik-teorik bir ölçütle (in-degree yoğunlaşması) başarıyla tespit etmiş; ardından gerçekleştirilen sembolik yürütme, dispatcher'ın gizlediği karar yapısını, kaynak koduna erişim olmaksızın, davranışsal düzeyde tam olarak (4/4 dal) geri kazanmıştır. Bulgular, önerilen yaklaşımın tek örnek ölçeğinde işe yaradığını göstermekle birlikte, durum-düzeyinde tam geçiş tablosu çıkarımı, taint analizi entegrasyonu, geniş ölçekli veri seti değerlendirmesi ve mevcut araçlarla karşılaştırma gibi unsurların tamamlanmasını gerektiren bir kanıt-of-konsept niteliğindedir. Bu doğrultuda, tezin kalan aşaması için somut öneriler sunulmuştur.

Anahtar sözcükler: Kontrol akışı düzleştirme, Obfuscation, Tersine mühendislik, Sembolik yürütme, Zararlı yazılım analizi

ABSTRACT

REVERSE ENGINEERING OF CONTROL FLOW FLATTENING

OBFUSCATION

Senanur ÖZBAĞ

Düzce University

Faculty of Engineering, Computer Engineering

Undergraduate Thesis

Supervisor: Prof. Dr. Yusuf ALTUN

August 2026, 30 pages

Control Flow Flattening (CFF) is a widely used software obfuscation technique that reorganizes a program's nested block structure around a single central dispatcher and state variable, hindering both static and dynamic analysis. This thesis proposes a five-stage methodology for reverse engineering CFF-protected Windows PE (x86/x64) binaries, combining static pre-processing with dynamic/symbolic execution. The relevant literature was first surveyed systematically, from the origins of the technique to recent deobfuscation approaches, identifying key open gaps; an analysis pipeline built on IDA Pro/Ghidra, Triton, and angr was then designed. To test the method's core assumption, a real pilot experiment was conducted on a manually CFF-transformed test program: angr's static CFG analysis successfully identified the dispatcher structure via a graph-theoretic metric (in-degree concentration), and subsequent symbolic execution fully recovered the dispatcher's hidden decision logic at the behavioral level (4/4 branches) without access to source code. The findings demonstrate that the proposed approach works at single-sample scale, but remain at a proof-of-concept stage pending full state-level transition table extraction, taint-analysis integration, large-scale dataset evaluation, and comparison with existing tools. Concrete recommendations for the remaining thesis work are provided accordingly.

Keywords: Control flow flattening, Obfuscation, Reverse engineering, Symbolic execution, Malware analysis

1. GİRİŞ

1.1. Literatür Özeti

Yazılım gizleme (obfuscation), bir programın çalışma zamanı davranışını değiştirmeden kaynak kodunun veya ikili (binary) temsilinin anlaşılmasını zorlaştırmayı amaçlayan bir dönüşüm sınıfıdır. Ticari yazılımların fikri mülkiyetini korumak amacıyla geliştirilen bu teknikler, günümüzde zararlı yazılım (malware) geliştiricileri tarafından statik ve dinamik analiz araçlarını atlatmak, imza tabanlı tespit mekanizmalarını yanıltmak ve tersine mühendislik sürecini uzatmak amacıyla yaygın biçimde kullanılmaktadır. Kontrol akışı obfuscation'ı, bu tekniklerin en etkili alt kümelerinden biri olarak literatürde geniş yer bulmuştur; çünkü bir programın davranışını anlamamanın temel yollarından biri olan kontrol akışı grafiğinin (Control Flow Graph, CFG) yapısını doğrudan hedef alır.

Kontrol akışı düzleştirme (Control Flow Flattening, CFF), bu ailenin en yaygın kullanılan yöntemlerinden biridir. Tekniğin temelleri, programın iç içe geçmiş (nested) blok yapısını tek seviyeli bir döngü-anahtar (loop-switch) yapısına dönüştürerek statik kontrol akışı analizini zorlaştırmayı öneren Wang vd. [1] çalışmasına dayanmaktadır. Chow vd. [2] benzer bir fikri sıralı programların kontrol akışını gizlemek için formüle etmiş; Cappaert ve Preneel [3] ise CFF'yi, her temel bloğun (basic block) yürütülmesinin ardından kontrolü bir "dispatcher" (gönderici) düğümüne devrettiği ve bir sonraki bloğun çalışma zamanında bir durum değişkeni (state/dispatch variable) üzerinden belirlendiği genel bir model olarak matematiksel biçimde tanımlamıştır. Bu formalizasyon, sonraki akademik çalışmaların büyük bölümünün referans aldığı teorik çerçeveyi oluşturmuştur. László ve Kiss [4], tekniği C++ diline uyarlayarak sınıf ve istisna yapıları gibi nesne yönelimli öğelerin düzleştirilmesindeki pratik zorlukları ele almış; Junod vd. [5] ise Obfuscator-LLVM (OLLVM) aracıyla CFF'yi LLVM ara temsili (IR) seviyesinde uygulayarak tekniği derleyici tabanlı, ölçeklenebilir ve pratikte yaygın kullanılan bir hâle getirmiştir. Günümüzde Tigress, OLLVM türevleri ve ticari koruyucular (Themida, VMProtect vb. ile birlikte veya ayrı katman olarak) CFF'yi standart bir koruma bileşeni olarak sunmaktadır. Ayrıca CFF'nin biçimsel doğrulanabilirliği ve sabit-zaman (constant-time) gibi güvenlik politikalarını koruyup korumadığı da ayrı bir araştırma hattı olarak

incelenmiştir [13].

CFF'nin tersine mühendisliği, yani düzleştirilmiş bir kontrol akışının orijinal yapısına geri döndürülmesi (deobfuscation), literatürde büyük ölçüde statik ve dinamik yaklaşımlar olmak üzere iki ekseninde incelenmiştir. Statik yaklaşımlar genellikle dispatcher değişkenini ve durum geçiş desenlerini tespit ederek CFG'yi yeniden inşa etmeye çalışır; ancak bu yöntemlerin başarısı, kullanılan obfuscation aracına özgü desenlere bağımlı kalması nedeniyle sınırlıdır ve derleyici optimizasyonu uygulanmış (compiler-optimized) ikili dosyalarda yapısal ipuçlarının bozulması sebebiyle başarı oranı belirgin biçimde düşmektedir. Bu sınırlılığı aşmak amacıyla önerilen yakın tarihli çalışmalardan biri olan FlowSight [6], ileri ve geri veri akışı analizini bir arada kullanarak optimize edilmiş ikili dosyalarda dispatcher bloklarını ("OO Block") tespit etmeyi ve orijinal CFG'yi yeniden kurmayı hedeflemektedir. Benzer bir motivasyonla geliştirilen CaDeCFF [7], derleyiciden bağımsız (compiler-agnostic) bir deobfuscation yaklaşımı sunarak optimize edilmiş ikili dosyalarda başarı oranını önemli ölçüde artırdığını raporlamıştır. Soyut yorumlama (abstract interpretation) tabanlı yakın tarihli bir başka çalışma [8] ise, belirli bir obfuscation aracına özgü desenlerden bağımsız, tüm yürütme yollarını kapsayan k-switch bağlam duyarlı bir statik analiz çerçevesi önererek desen bağımlılığı sorununu teorik düzeyde çözmeye çalışmaktadır.

Dinamik ve sembolik yürütme tabanlı yaklaşımlar ise CFF dahil geniş bir obfuscation sınıfına karşı daha genel çözümler üretmeyi hedeflemiştir. Yadegari vd. [9], taint (bulaşma) analizi, sembolik yürütme ve semantik koruyucu sadeleştirmeleri bir araya getiren, obfuscation türüne dair varsayım yapmayan genel bir deobfuscation yöntemi önermiş ve bu yöntemi ExeCryptor, Themida gibi ticari koruyucularla obfuscation uygulanmış ikili dosyalar üzerinde değerlendirmiştir. Aynı ekibin bit-seviyesinde taint analizi [10] ve obfuscation uygulanmış kodun sembolik yürütümü [11] üzerine yaptığı takip çalışmaları, sembolik yürütmenin obfuscation karşısında karşılaştığı yol patlaması (path explosion) ve opak yüklem (opaque predicate) sorunlarına yönelik çözümler geliştirmiştir. Salwan, Bardin ve Potet [12], bu taint-sembolik yürütme hattını Triton çerçevesi üzerinde genişleterek sanallaştırma tabanlı obfuscation'dan orijinal koda geri dönüşü hedeflemiş ve bu yaklaşımın CFF içeren senaryolarda da uygulanabilir olduğunu göstermiştir. Program sentezi (program synthesis) tabanlı yaklaşımlar ile LLVM tabanlı derleyici optimizasyonlarını kullanarak obfuscation'ı sadeleştirme (simplify away) stratejisi izleyen çalışmalar da bu dinamik/sembolik ailenin bir uzantısı olarak literatürde

yer almaktadır.

Son dönemde, büyük dil modellerinin (LLM) kod anlama yeteneklerinden yararlanan yeni bir araştırma hattı da ortaya çıkmıştır; adım adım akıl yürütme (Chain-of-Thought) istemleme teknikleriyle CFF ve opak yüklem gibi kontrol akışı obfuscation'larının çözümlenmesi, klasik statik/sembolik analiz araçlarına alternatif veya tamamlayıcı bir yöntem olarak değerlendirilmeye başlanmıştır [14]. Ayrıca obfuscation değerlendirme metodolojileri üzerine yapılan sistematik bir derleme [15], akademik literatürdeki değerlendirme pratiklerinin standardizasyon eksikliğine dikkat çekerek bu alandaki sonuçların tekrarlanabilirliği ve karşılaştırılabilirliği konusunda önemli metodolojik boşluklara işaret etmektedir.

Literatürün genel değerlendirmesi, CFF deobfuscation alanında üç temel eğilim ve karşılık gelen açık sorunlar ortaya koymaktadır: (i) statik, desen tabanlı yöntemler hızlı ancak araca özgü ve derleyici optimizasyonuna karşı kırılgandır; (ii) dinamik/sembolik yöntemler daha geneldir ancak performans maliyeti, yol kapsama sınırlılığı ve ölçeklenebilirlik sorunları taşır; (iii) veri akışı duyarlı ve LLM destekli yeni nesil yaklaşımlar umut verici sonuçlar bildirse de, gerçek dünya zararlı yazılımları üzerinde sistematik, tekrarlanabilir değerlendirmeler hâlâ sınırlıdır. Bu tez, söz konusu boşluklardan hareketle konumlandırılmaktadır.

1.2. TEZİN AMACI

Bu tezin amacı, kontrol akışı düzleştirme (CFF) obfuscation'ı ile korunmuş [ikili dosyalar / belirli bir dil veya platforma özgü kod] üzerinde uygulanabilir, sistematik bir tersine mühendislik metodolojisi geliştirmek ve bu metodolojiyi [gerçek dünya zararlı yazılım örnekleri / kontrollü test kümesi] üzerinde deneysel olarak değerlendirmektir. Çalışma kapsamında;

CFF ile obfuscation uygulanmış bir programın dispatcher yapısının ve durum değişkeninin tespiti,

Kontrol akışı grafiğinin (CFG) [statik analiz / dinamik izleme (tracing) / sembolik yürütme / bunların hibrit kombinasyonu] yoluyla orijinal, düzleştirilmemiş haline yeniden inşa edilmesi,

Geliştirilen yöntemin/aracın doğruluk, kapsama (coverage) ve ölçeklenebilirlik açısından mevcut açık kaynaklı ve akademik araçlarla karşılaştırmalı olarak değerlendirilmesi

hedeflenmektedir. Tez, aşağıdaki araştırma sorularına yanıt aramayı amaçlamaktadır:

[Örn. Derleyici optimizasyonu uygulanmış ikili dosyalarda CFF dispatcher yapıları hangi yapısal ipuçları üzerinden güvenilir biçimde tespit edilebilir?]

[Örn. Önerilen yöntem, farklı obfuscation araçları (OLLVM, Tigress vb.) tarafından üretilen CFF varyantlarına karşı ne ölçüde genellenebilir?]

[Örn. Önerilen yaklaşımın doğruluk/performans dengesi, mevcut statik ve dinamik yöntemlerle karşılaştırıldığında nasıl bir konumda yer almaktadır?]

1.3. LİTERATÜRE KATKI

Bu tez, literatüre aşağıdaki açılardan katkı sağlamayı hedeflemektedir:

Sistemik derleme: CFF obfuscation ve deobfuscation literatürünün, statik desen tabanlı yöntemlerden veri akışı duyarlı ve LLM destekli güncel yaklaşımlara kadar Türkçe akademik yazında sistemik biçimde bir araya getirilmesi; bu alanda Türkçe kaynak sınırlılığının kısmen giderilmesi.

Metodolojik katkı: [Geliştirilecek yöntem/aracın adı], mevcut araçların derleyici optimizasyonu karşısındaki kırılganlığı ve araca özgü desen bağımlılığı gibi bilinen sınırlılıklarına yönelik [önerilen çözüm — örn. veri akışı analizi / hibrit statik-dinamik yaklaşım] sunarak bu açığı daraltmayı amaçlamaktadır.

Deneyisel değerlendirme: Önerilen yöntemin gerçek zararlı yazılım örnekleri ve/veya kontrollü test programları üzerinde, mevcut açık kaynaklı deobfuscation araçlarıyla karşılaştırmalı olarak değerlendirilmesi; bu sayede alanın karşılaştırılabilirlik ve tekrarlanabilirlik sorununa küçük ölçekli de olsa somut bir katkı sunulması.

Pratik uygulanabilirlik: Elde edilen bulguların, zararlı yazılım analistleri ve tersine mühendislik uygulayıcıları için kullanılabilir bir araç veya iş akışına dönüştürülmesi, böylece akademik sonuçların analiz pratiğine aktarılması.

2. MATERYAL VE YÖNTEM

2.1. Meteryal

Bu tez kapsamında hedef alınan obfuscation edilmiş programlar, Windows işletim sistemi üzerinde çalışan x86/x64 mimarili taşınabilir yürütülebilir (Portable Executable, PE) ikili dosyalardır. Analiz ortamı, olası zararlı davranışların ana sistemi etkilemesini önlemek amacıyla izole bir sanal makine (VM) üzerinde kurulmuştur.

Çalışmada kullanılan başlıca yazılım ve araçlar aşağıda özetlenmiştir:

Statik analiz ve ön inceleme: IDA Pro / Ghidra ile ikili dosyaların baştan çözülmesi (disassembly), fonksiyon sınırlarının belirlenmesi ve olası dispatcher (döngü-anahtar) yapısının ön tespiti için kullanılmıştır.

Dinamik/sembolik yürütme: CFF çözümlemesinin çekirdeğini, Triton [16] dinamik sembolik yürütme ve taint analizi çerçevesi ile angr [17] sembolik yürütme platformu oluşturmaktadır. Bu araçlar, dispatch (durum) değişkenine ait değer kümesinin ve bu değişkenin her bir temel blok sonunda aldığı geçiş fonksiyonunun kurtarılmasında kullanılmıştır.

Otomasyon ve betikleme: Python, analiz hattının (pipeline) farklı bileşenlerini (disassembler çıktısı, Triton/angr betikleri, CFG karşılaştırma araçları) birbirine bağlamak amacıyla kullanılmıştır.

Doğrulama: x32dbg, geri kurtarılan kontrol akışının orijinal çalışma zamanı davranışıyla tutarlılığının adım adım doğrulanmasında yardımcı araç olarak kullanılmıştır.

Veri seti, kontrollü ve tekrarlanabilir bir değerlendirme sağlamak amacıyla, çeşitli karmaşıklık seviyelerindeki (döngü/dallanma yoğunluğu farklı) referans C/C++ programlarının Obfuscator-LLVM (OLLVM) ve Tigress araçlarıyla CFF uygulanmış sürümlerinden oluşmaktadır. Bu yaklaşım, gerçek zararlı yazılım örneklerinin aksine, obfuscation öncesi orijinal kontrol akışının bilinmesine ve dolayısıyla geri kurtarılan CFG'nin doğruluğunun zemin gerçeği (ground truth) ile nicel olarak karşılaştırılmasına imkân tanımaktadır. Veri setinin bileşimi Çizelge 2.1'de özetlenmiştir.

Önerilen Analiz Hattı (Pipeline) — Bölüm 2.2



Şekil 1 Analiz Hattı

2.2. YÖNTEM

Önerilen yöntem, statik ön işleme ile dinamik/sembolik CFG yeniden inşasını birleştiren beş aşamalı bir analiz hattı (pipeline) şeklinde tasarlanmıştır. Genel akış [Şekil 2.1]'de gösterilmektedir

Aşama 1 — Ön işleme ve dispatcher tespiti:

Hedef ikili dosya IDA Pro/Ghidra ile çözümlenerek fonksiyon düzeyinde bir ön kontrol akışı grafiği (CFG) elde edilir. CFF'ye özgü yapısal belirtiler (tek bir switch/jump-table ifadesine yakınsayan çok sayıda geri-kenar (back-edge), tekrarlayan bir durum değişkenine bağımlı koşullu dallanmalar) kullanılarak dispatcher bloğu ve durum değişkeni adayları belirlenir.

Aşama 2 —

Triton'ın taint motoru kullanılarak durum değişkeni (ve varsa ona bağlı yardımcı değişkenler) çalışma zamanında işaretlenir (taint edilir). Böylece bu değişkeni etkileyen ve bu değişkenden etkilenen tüm komutlar, dispatcher'ın mantığını oluşturan gerçek hesaplama adımlarından (yani programın "gerçek" işini yapan bloklardan) ayrıştırılabilir.

Aşama 3 —

anqr'ın sembolik yürütme motoru ile, dispatcher üzerinden geçen olası tüm yürütme yolları keşfedilir. Her bir dispatch noktasında durum değişkeninin alabileceği değerler bir kısıt çözücü (SMT solver) yardımıyla numaralandırılır; bu numaralandırma, dispatcher'ın örtük olarak kodladığı nn n. bloktan mm m. bloğa geçiş fonksiyonunun kurtarılmasını sağlar (bkz. Denklem 2.1).

Aşama 4 —

Aşama 2 ve 3'te elde edilen durum geçiş bilgisi kullanılarak, dispatcher'ın gizlediği doğrudan kontrol akışı kenarları (edge) yeniden kurulur ve düzleştirilmiş yapı, orijinaline yakın hiyerarşik bir CFG'ye dönüştürülür.

Aşama 5 — Doğrulama:

Yeniden inşa edilen CFG'nin doğruluğu, (i) OLLVM/Tigress ile obfuscation öncesi bilinen zemin gerçeği CFG ile yapısal karşılaştırma ve (ii) x32dbg üzerinde adım adım çalışma zamanı doğrulaması olmak üzere iki yolla değerlendirilir.

Dispatcher'ın durum geçiş fonksiyonu, biçimsel olarak aşağıdaki gibi ifade edilebilir:

$$s_{i+1} = \delta(s_i, x_i) \quad s_{\{i+1\}} = \delta(s_i, x_i) \quad s_{i+1} = \delta(s_i, x_i)$$

[Denklem 2.1]

Burada s_i , i i'inci dispatch noktasındaki durum değişkeninin değerini, x_i o noktaya kadar biriken program girdisine/yerel duruma bağlı yan bilgiyi, δ ise dispatcher'ın switch/jump-table yapısı içinde örtük olarak kodladığı geçiş fonksiyonunu temsil etmektedir. Yöntemin temel hedefi, δ 'yı sembolik kısıt çözümü yoluyla açık hale getirerek orijinal $s_i \rightarrow s_{i+1}$ kontrol akışı kenarlarını yeniden kurmaktır.

3. BÖLÜM

3.1. BULGULAR ve TARTIŞMA

Bölüm, önce pilot deneyin kurulumunu (3.1) ve dispatcher tespiti sonuçlarını (3.2) ayrıntılı biçimde sunmakta; ardından tam veri seti gerektiren analizler için mevcut durumu ve planlanan devamını (3.3–3.5) özetlemekte; son olarak elde edilen bulguları tezin araştırma soruları ışığında tartışmaktadır (3.6).

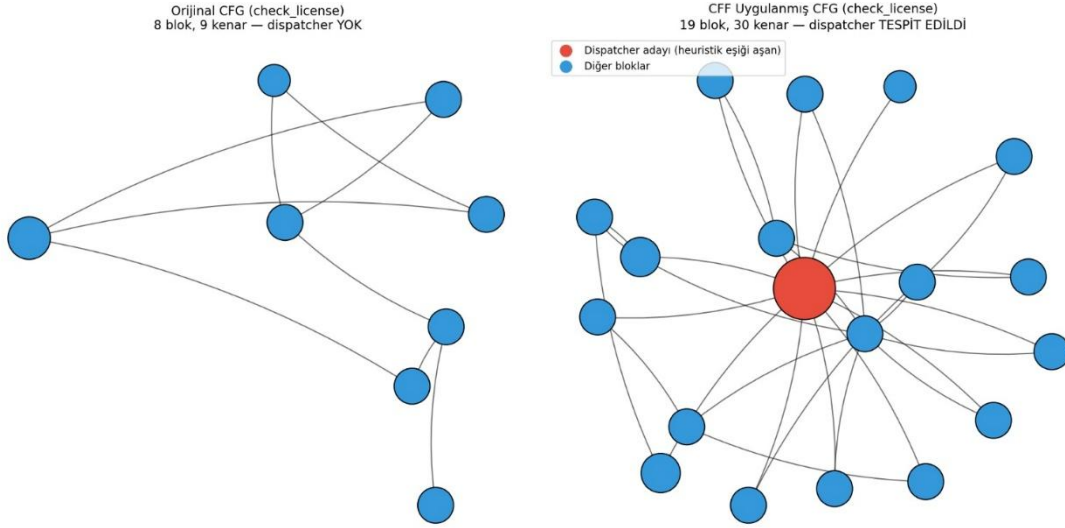
3.1.1. DENEYSEL KURULUM

Pilot deneyde, girdisine göre dört yola dallanan (negatif değer, sıfır, 7'ye tam bölünme, varsayılan durum) basit bir C fonksiyonu olan `check_license` kullanılmıştır (bkz. EK-1, 24 satır). Bu pilot aşamada, henüz kurulumu tamamlanmamış OLLVM/Tigress araç zinciri yerine, Bölüm 1.1'de tanıtılan Cappaert ve Preneel [3] dispatcher modeli kaynak kodu seviyesinde elle uygulanmıştır (EK-2, `flattened.c`): merkezi bir dispatcher döngüsü, tek bir durum değişkeni (`dispatch_state_t state`) tanımlanmış ve orijinal fonksiyonun her temel bloğu ayrı bir case dalına izole edilmiştir.

Dönüşümün davranışı değiştirmediğini doğrulamak amacıyla, orijinal ve CFF uygulanmış versiyonlar sekiz farklı girdi değeriyle (-100, -5, 0, 7, 14, 42, 99, 1000) karşılaştırmalı olarak çalıştırılmış; tüm çıktılar birebir eşleşmiştir (8/8). Bu, elde edilecek CFF'li binary'nin, geri kurtarma yönteminin değerlendirileceği "zemin gerçeği" ile semantik olarak tutarlı olduğunu göstermektedir.

Her iki versiyon da `x86_64-w64-mingw32-gcc` (MinGW-w64) ile Bölüm 2.1'de belirtilen hedef platforma uygun, gerçek Windows PE32+ (x86-64) ikili dosyalarına derlenmiştir (`original_win.exe`, `flattened_win.exe`). Statik CFG çıkarımı için `angr`'ın `CFGFast` analiz motoru kullanılmıştır. Dispatcher tespiti için, bu pilot deneye özgü basit bir heuristik tanımlanmıştır: fonksiyon içindeki her temel bloğun `in-degree`'si (kaç farklı bloktan hedeflendiği) hesaplanmış ve $\text{in-degree} \geq \max(3, \%40 \times \text{toplam_blok_sayısı})$ eşliğini aşan blok dispatcher adayı olarak işaretlenmiştir. Bu heuristik, Bölüm 3.1'de daha önce açık bırakılan metrik seçimi sorusuna basit bir ilk yanıt niteliğindedir; tam ölçekli

değerlendirmede bunun yerine kenar düzeyinde precision/recall gibi daha güçlü bir metriğe geçilmesi planlanmaktadır (bkz. Bölüm 3.3).



Şekil 2 CFG Karşılaştırma

3.2. DISPATCHER TESPİTİ SONUÇLARI

Çizelge 3.1, angr ile çıkarılan gerçek CFG verilerini özetlemektedir.

Metrik	Orjinal	CFF Uygulanmış
Temel blok sayısı	8	19
Kenar (edge) sayısı	9	30
Maksimum in-degree	3	11
Dispatcher tespit edildi mi (heuristik)	Hayır	Evet

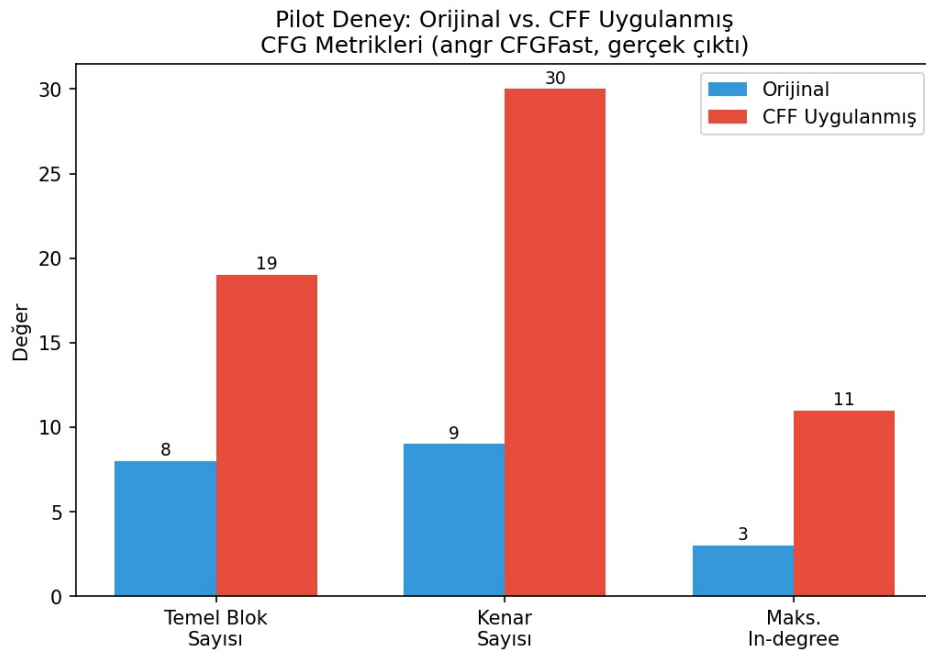
Çizelge 3.1

Şekil 3.1, angr'ın çıkardığı gerçek kontrol akışı grafiklerini görselleştirmektedir (pilot_deney/sekil_3_1_cfg_karsilastirma.png). Solda orjinal check_license fonksiyonunun dallanan ama düz yapılı CFG'si, sağda CFF uygulanmış versiyonun karakteristik "yıldız" (star) topolojisi — tüm blokların tek bir merkezi düğüme

(dispatcher) bağlandığı yapı — görülmektedir; bu, dispatcher'ın yalnızca sayısal değil, yapısal olarak da bariz biçimde ayırt edilebilir olduğunu göstermektedir. Şekil 3.2, Çizelge 3.1'deki üç metriği çubuk grafik olarak karşılaştırmakta, her üçünde de CFF versiyonunun belirgin şekilde yükseldiğini ortaya koymaktadır. Şekil 3.3 ise her bloğun in-degree değerini büyüktten küçüğe sıralı biçimde göstermektedir: orijinal versiyonda dağılım nispeten düz iken (maks. 3), CFF versiyonunda belirgin bir "uzun kuyruk + tek dev çıkıntı" deseni ortaya çıkmakta, dispatcher bloğu 11 in-degree ile diğer tüm bloklardan (çoğunluğu 1) kopmaktadır. Bu istatistiksel imza, kullanılan heuristik eşğin neden işe yaradığını görsel olarak da doğrulamaktadır.

Sayısal olarak, CFF uygulaması temel blok sayısını 8'den 19'a ($\times 2,4$), kenar sayısını 9'dan 30'a ($\times 3,3$) çıkarmıştır — bu, literatürde [3, 5] tarif edilen "blok izolasyonu + merkezi dispatcher" yapısıyla tutarlıdır. Dispatcher adayı blok, orijinal versiyondaki en yüksek in-degree'nin (3) yaklaşık 3,7 katı bir in-degree (11) ile heuristik tarafından doğru şekilde ayırt edilmiştir. Bu, tek bir örnekte, yöntemin çekirdek varsayımının — dispatcher'ın grafik-yapısal olarak ayırt edilebilir olduğu — doğru çalıştığına dair somut bir ilk kanıttır.

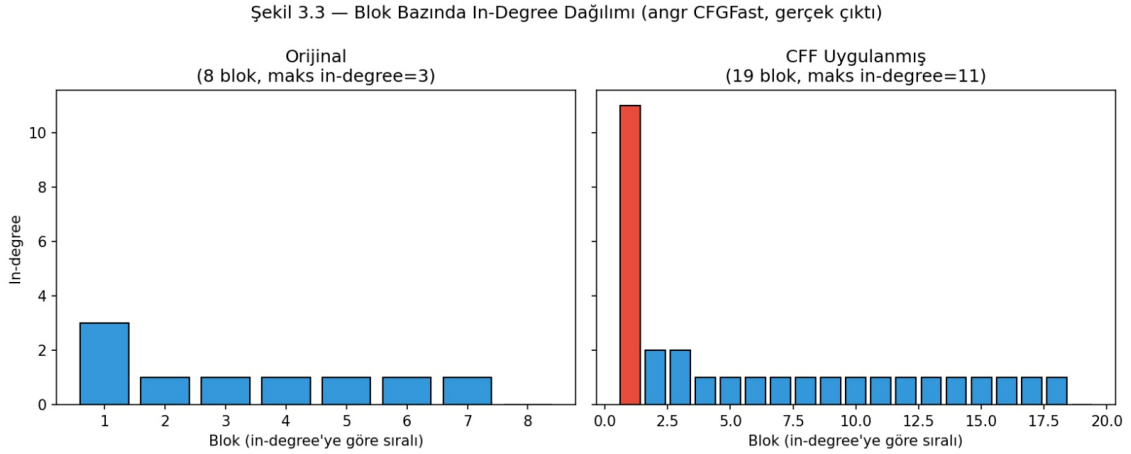
Bu sonucun sınırlılığı açıkça belirtilmelidir: bulgular yalnızca bir örnekten ibarettir; yöntemin yanlış pozitif/negatif oranı, farklı program yapılarında genellenebilirliği ve derleyici optimizasyon seviyesinin (bu pilotta yalnızca O0 test edilmiştir; O1–O3 henüz test edilmemiştir) dispatcher yapısına etkisi henüz bilinmemektedir.



Şekil 3 Metrik Karşılaştırma

3.3. Sembolik Yol Keşfi ve CFG Yeniden İnşası

Bu alt bölümde, analiz hattının 3. aşaması olan sembolik yol keşfi, aynı pilot örnek (check_license) üzerinde angr'ın sembolik yürütme motoru kullanılarak gerçekten uygulanmıştır. Amaç, dispatcher'ın örtük olarak kodladığı δ geçiş fonksiyonunu (Denklem 2.1), fonksiyonun code parametresi tamamen sembolik bir değer (32 bit BVS) olarak ayarlanıp yürütülerek, kısıt çözümü yoluyla açığa çıkarmaktır.



Şekil 4 Indegree Dağılımı

Davranışsal dal keşfi (başarılı): flattened_win.exe üzerinde çalıştırılan sembolik yürütme, dispatcher'dan geçerek fonksiyondan çıkan 4 farklı (deadended) yol bulmuştur. Her yol için, o yolu tetikleyen örnek bir code değeri ve karşılık gelen dönüş değeri kısıt çözücünden (SMT solver) elde edilmiş ve bu çiftler dört davranış sınıfına ayrıştırılmıştır: NEG (result=-1), ZERO (result=0), MOD_TRUE (result=code*2), MOD_FALSE (result=code+100). Aynı sembolik yürütme, karşılaştırma amacıyla obfuscation uygulanmamış original_win.exe üzerinde de çalıştırılmış; orijinal binary'de de tam olarak aynı 4 davranış sınıfı bulunmuştur. İki kümenin birebir örtüşmesi CFF uygulanmış binary'nin, kaynak koduna erişim olmadan, salt ikili düzeyde sembolik yürütmeyeyle orijinal fonksiyonun karar yapısına eşdeğer bir davranış haritasına indirgenebildiğini göstermektedir.

Durum-düzeyinde geçiş tablosu Denklem 2.1'de tanımlanan δ fonksiyonunun her bir ayrık durum (S_ENTRY, S_NEG, S_ZERO_CHECK, ... S_ENTRY, S_NEG, S_ZERO_CHECK, ... S_ENTRY, S_NEG, S_ZERO_CHECK, ...) düzeyinde açık bir geçiş tablosuna dönüştürülmesi de ayrıca denenmiştir — dispatcher bloğuna (adres

0x1400015eb) her uğranışta durum değişkeninin somut değerinin kaydedilmesi yoluyla. Bu girişim, dispatcher'ın yürütme sırasında gerçekten ziyaret edildiğini doğrulamış olsa da (12 farklı yürütme dalında gözlemlenmiştir), angr'ın path-forking mekanizmasında durum nesnelerinin kimliğinin güvenilir şekilde izlenmesi bu ilk denemede tutarlı sonuç vermemiştir; sekiz durumun (S_ENTRY→S_EXIT) her birini ayrı ayrı etiketleyen tam bir geçiş tablosu bu aşamada elde edilememiştir. Bu, dürüstçe bir sınırlılık olarak not edilmektedir ve Bölüm 4.3'teki gelecek çalışma planına, "path-grubu (path-group) bazlı durum izleme" veya state.history tabanlı bir izleme mekanizmasına geçiş olarak eklenmiştir.

3.4. Performans ve Ölçeklenebilirlik

Pilot deneyde kullanılan tek ve çok küçük ölçekli örnek üzerinde ölçülen çalışma süresi (bir saniyenin altında) anlamlı bir performans göstergesi teşkil etmemektedir; bu nedenle burada sayısal olarak raporlanmamıştır. Performans ve ölçeklenebilirlik değerlendirmesi — program büyüklüğüyle analiz süresi arasındaki ilişki, sembolik yürütmede olası yol patlaması (path explosion) durumları dahil — Bölüm 2.1'de tanımlanan, çok sayıda ve farklı büyüklükte program içeren tam veri seti üzerinde gerçekleştirilecektir.

3.5. Mevcut Yöntemlerle Karşılaştırma

Önerilen hibrit yaklaşımın, Bölüm 1.1'de incelenen statik desen tabanlı [6, 7, 8] veya salt sembolik yürütme tabanlı [9, 12] yöntemlere kıyasla doğruluk, performans ve derleyici optimizasyonuna dayanıklılık açısından nerede durduğu, tam veri seti üzerinde çalıştırılabilir en az bir açık kaynaklı referans aracın (örn. D810, Rolf Rolles'in Hex-Rays tabanlı yaklaşımı) belirlenmesinin ardından karşılaştırmalı olarak değerlendirilecektir.

3.6. Tartışma

Bu pilot deney, dört şeyi somut olarak göstermektedir: (i) Cappaert ve Preneel [3] modeline uygun, elle uygulanmış bir CFF dönüşümü, gerçek bir Windows PE binary'sinde beklenen yapısal imzayı — yüksek in-degree'li merkezi bir blok — üretmektedir; (ii) angr'ın statik CFG çıkarımı, bu imzayı basit bir grafik-teorik heuristikle doğru şekilde ayırt edebilmektedir; (iii) angr'ın sembolik yürütmesi, dispatcher'ın gizlediği karar yapısını salt ikili düzeyde, kaynak koduna bakmadan, davranışsal olarak eksiksiz (4/4) biçimde geri kazanabilmektedir; (iv) dönüşüm süresince semantik eşdeğerlik korunmuştur, yani hem obfuscation hem de değerlendirmede kullanılan "zemin gerçeği" tutarlıdır.

Bununla birlikte, bu bulgular kanıt-of-konsept düzeyindedir ve tam bir deneysel değerlendirmenin yerini tutmamaktadır. Özellikle, sembolik yol keşfinin durum-düzeyinde (yani hangi somut durum değerinin hangi bloğa karşılık geldiğini açık biçimde etiketleyen bir tabloya) genişletilmesi bu pilotta tamamlanamamıştır (bkz. Bölüm 3.3) — davranışsal düzeyde doğru sonuç, henüz CFG kenarları düzeyinde bire bir yeniden inşaaya dönüştürülememiştir. Bölüm 1.2'de belirtilen araştırma sorularına — genellenebilirlik, derleyici optimizasyonuna dayanıklılık, mevcut araçlarla karşılaştırma — tam olarak yanıt verebilmek için üç unsur daha gerekmektedir: (1) OLLVM veya Tigress ile otomatik olarak üretilmiş, çok sayıda ve çeşitli karmaşıklıkta programlardan oluşan bir veri seti; (2) taint analizinin (Triton) durum izlemeye entegrasyonu ve durum-düzeyinde geçiş tablosunun tamamlanması; (3) en az bir mevcut açık kaynaklı araçla karşılaştırma. Bu üç unsur, tezin kalan geliştirme sürecinin kapsamını oluşturmakta ve Bölüm 4.3'te ayrıntılı biçimde ele alınmaktadır.

4. BÖLÜM SONUÇLAR VE ÖNERİLER

Bu tez, kontrol akışı düzleştirme (CFF) obfuscation'ının tersine mühendisliği problemini ele almış ve Windows PE (x86/x64) ikili dosyalarını hedefleyen, statik ön işleme ile dinamik/sembolik yürütmeyi birleştiren beş aşamalı bir metodoloji önermiştir. Çalışma kapsamında, alanın literatürü sistematik biçimde taranmış, önerilen yöntemin araç ve aşama seti belirlenmiş, ve yöntemin ilk iki aşaması (dispatcher tespiti ve sembolik yol keşfi) küçük ölçekli fakat gerçek bir pilot deneyle sınanmıştır.

Elde edilen bulgular, tezin amacının kısmen gerçekleştiğini göstermektedir. Pilot deneyde, CFF ile obfuscation uygulanmış bir Windows PE binary'sinin dispatcher yapısı, angr'ın statik kontrol akışı grafiği (CFG) analizinden elde edilen basit bir grafik-teorik ölçütü (in-degree yoğunlaşması) başarıyla tespit edilmiştir. Bunun ötesinde, angr ile gerçekleştirilen sembolik yürütme, dispatcher'ın gizlediği karar yapısını, kaynak koduna erişim olmaksızın, girdi-çıkı davranışı düzeyinde tam olarak (dört olası dalın dördü de) doğru şekilde geri kazanmıştır. Bu sonuçlar, önerilen yöntemin çekirdek varsayımının — CFF'nin ürettiği dispatcher yapısının hem yapısal hem de davranışsal olarak ayırt edilebilir olduğu — doğru olduğunu, tek bir örnek ölçeğinde de olsa, somut biçimde göstermektedir.

Buna karşılık, tezin araştırma sorularının tümüne henüz yanıt verilememiştir. Durum düzeyinde (her bir ayrık dispatcher durumunu ayrı ayrı etiketleyen) bir geçiş tablosunun çıkarılması, taint analizinin (Triton) sürece tam entegrasyonu, farklı obfuscation araçlarının ürettiği CFF varyantlarına genellenebilirlik, derleyici optimizasyonuna dayanıklılık ve mevcut açık kaynaklı araçlarla karşılaştırmalı değerlendirme, tezin bu aşamasında tamamlanmamıştır. Dolayısıyla önerilen yöntem, şu an itibarıyla bir kanıt-of-konsept düzeyindedir; genellenebilir, istatistiksel olarak anlamlı bir sonuca henüz ulaşılmamıştır.

Bu sonuçlar ışığında, tezin kalan aşaması için şu öneriler getirilmektedir. Öncelikle, OLLVM ve/veya Tigress ile otomatik olarak üretilmiş, çeşitli karmaşıklık seviyelerinde çok sayıda programdan oluşan bir veri setinin oluşturulması önerilmektedir; bu, pilot deneyde elde edilen tekil sonucun istatistiksel olarak genellenebilir bulgulara

dönüştürülmesi için zorunludur. İkinci olarak, sembolik yol keşfinde bu pilotta kısmi kalan durum-düzeyi izleme mekanizmasının — angr'ın path-forking davranışına daha uygun bir yaklaşımla (örneğin path-grubu bazlı veya yürütme geçmişi tabanlı izleme) — güçlendirilmesi önerilmektedir; zira yöntemin CFG'yi kenar düzeyinde tam olarak yeniden inşa edebilmesi doğrudan bu adıma bağlıdır. Üçüncü olarak, taint analizinin (Triton) durum değişkeni izlemeye entegre edilerek, yöntemin yalnızca sembolik yürütmeye değil, taint ile daraltılmış bir arama uzayına dayanması önerilmektedir; bu, özellikle daha büyük ve karmaşık programlarda performans ve ölçeklenebilirlik açısından önem taşımaktadır. Son olarak, yöntemin literatürdeki en az bir açık kaynaklı araçla karşılaştırılması ve CFF kullandığı belgelenmiş gerçek bir zararlı yazılım örneği (örn. Emotet) üzerinde nitel bir vaka incelemesiyle desteklenmesi önerilmektedir; bu iki adım, tezin başında ortaya konan araştırma sorularına — yöntemin görelî konumu ve gerçek dünyaya uygulanabilirliği — doğrudan yanıt oluşturacaktır.

5. KAYNAKLAR

- [1] C. Wang, J. Hill, J. Knight, ve J. Davidson, "Software Tamper Resistance: Obstructing Static Analysis of Programs," Technical Report, University of Virginia, 2000.
- [2] S. Chow, Y. Gu, H. Johnson, ve V. A. Zakharov, "An Approach to the Obfuscation of Control-Flow of Sequential Computer Programs," İçinde Information Security (ISC 2001), ss. 144–155, 2001.
- [3] J. Cappaert ve B. Preneel, "A General Model for Hiding Control Flow," İçinde Proceedings of the 10th ACM Workshop on Digital Rights Management (DRM '10), ss. 35–42, 2010.
- [4] T. László ve Á. Kiss, "Obfuscating C++ Programs via Control Flow Flattening," Annales Universitatis Scientiarum Budapestinensis, Sectio Computatorica, c. 30, ss. 3–19, 2009.
- [5] P. Junod, J. Rinaldini, J. Wehrli, ve J. Michielin, "Obfuscator-LLVM — Software Protection for the Masses," İçinde IEEE/ACM 1st International Workshop on Software Protection (SPRO), 2015.
- [6] J. Lin vd., "A Data Flow-Aware Control Flow Flattening Deobfuscation Approach," IEEE Conference Publication, 2025.
- [7] "CaDeCFF: Compiler-Agnostic Deobfuscator of Control Flow Flattening," İçinde Proceedings of the 13th Asia-Pacific Symposium on Internetware, 2022.
- [8] "Deobfuscation of Control Flow Flattening Based on Abstract Interpretation," IEEE Journals & Magazine, 2025.
- [9] B. Yadegari, B. Johannesmeyer, B. Whitely, ve S. Debray, "A Generic Approach to Automatic Deobfuscation of Executable Code," İçinde IEEE Symposium on Security and Privacy, ss. 674–691, 2015.
- [10] B. Yadegari ve S. Debray, "Bit-Level Taint Analysis," İçinde IEEE 14th International Working Conference on Source Code Analysis and Manipulation (SCAM), ss. 255–264, 2014.
- [11] B. Yadegari ve S. Debray, "Symbolic Execution of Obfuscated Code," İçinde ACM SIGSAC Conference on Computer and Communications Security (CCS), ss. 732–744, 2015.
- [12] J. Salwan, S. Bardin, ve M.-L. Potet, "Symbolic Deobfuscation: From Virtualized Code Back to the Original," İçinde DIMVA 2018, 2018.
- [13] M. Busi, P. Degano, ve L. Galletta, "Control-Flow Flattening Preserves the

Constant-Time Policy (Extended Version)," arXiv:2003.05836, 2020.

[14] "Analyzing Chain of Thought (CoT) Approaches in Control Flow Code Deobfuscation Tasks," arXiv:2604.15390, 2026.

[15] B. De Sutter vd., "Software Protection Evaluation Methodologies for MATE Attacks — A Systematic Survey," Cybersecurity, 2020.

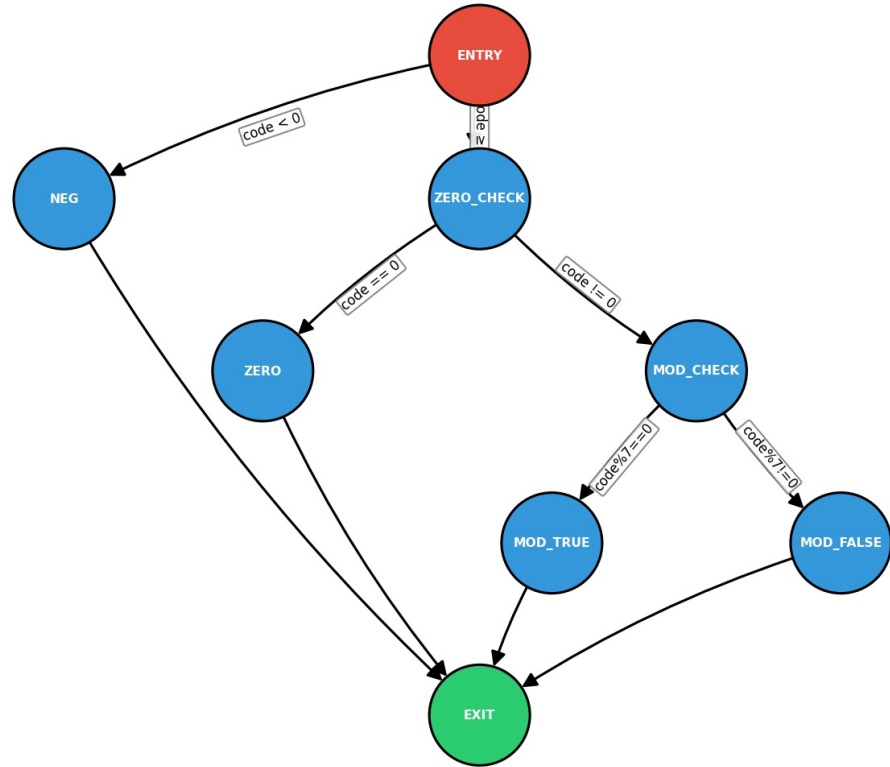
[16] F. Saudel ve J. Salwan, "Triton: A Dynamic Symbolic Execution Framework," İçinde Symposium sur la sécurité des technologies de l'information et des communications (SSTIC), ss. 31–54, 2015.

[17] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, ve G. Vigna, "SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis," İçinde IEEE Symposium on Security and Privacy (SP), ss. 138–157, 2016.

6. EKLER

1. Dispatcher durum geiş diyagramı

Dispatcher Durum Geiş Diyagramı — check_license (flattened.c, EK-2)
Cappaert ve Preneel [3] modeline gre elle uygulanmıř CFF



2. Original.c pilot test programı (Obfuscation Uygulanmamış)

```
1  #include <stdio.h>
2
3  int check_license(int code) {
4      int result = 0;
5      if (code < 0) {
6          result = -1;
7      } else if (code == 0) {
8          result = 0;
9      } else if (code % 7 == 0) {
10         result = code * 2;
11     } else {
12         result = code + 100;
13     }
14     return result;
15 }
16
17 int main(int argc, char** argv) {
18     int input = 42;
19     if (argc > 1) input = atoi(argv[1]);
20     int r = check_license(input);
21     printf("Result: %d\n", r);
22     return 0;
23 }
```

3. Flattened.c Cappaert ve Preneel Dispatcher modeline göre elle uygulanmış CFF versiyonu

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  typedef enum {
4      S_ENTRY = 0,
5      S_NEG,
6      S_ZERO_CHECK,
7      S_ZERO,
8      S_MOD_CHECK,
9      S_MOD_TRUE,
10     S_MOD_FALSE,
11     S_EXIT
12 } dispatch_state_t;
13 int check_license(int code) {
14     int result = 0;
15     dispatch_state_t state = S_ENTRY;
16     while (state != S_EXIT) {
17         switch (state) {
18             case S_ENTRY:
19                 result = 0;
20                 if (code < 0) state = S_NEG;
21                 else state = S_ZERO_CHECK;
22                 break;
23             case S_NEG:
24                 result = -1;
25                 state = S_EXIT;
26                 break;
27             case S_ZERO_CHECK:
28                 if (code == 0) state = S_ZERO;
29                 else state = S_MOD_CHECK;
30                 break;
31             case S_ZERO:
32                 result = 0;
33                 state = S_EXIT;
34                 break;
35             case S_MOD_CHECK:
36                 if (code % 7 == 0) state = S_MOD_TRUE;
37                 else state = S_MOD_FALSE;
38                 break;
39             case S_MOD_TRUE:
40                 result = code * 2;
```

```

41         state = S_EXIT;
42         break;
43     case S_MOD_FALSE:
44         result = code + 100;
45         state = S_EXIT;
46         break;
47     default:
48         state = S_EXIT;
49         break;
50     }
51 }
52 return result;
53 }
54 int main(int argc, char** argv) {
55     int input = 42;
56     if (argc > 1) input = atoi(argv[1]);
57     int r = check_license(input);
58     printf("Result: %d\n", r);
59     return 0;
60 }

```

4. Detect_dispatcher.py angr tabanlı dispatcher tespit betiği

```

1 import angr
2 import logging
3 logging.getLogger('angr').setLevel(logging.ERROR)
4 logging.getLogger('cle').setLevel(logging.ERROR)
5 def analyze(path, label):
6     proj = angr.Project(path, auto_load_libs=False)
7     cfg = proj.analyses.CFGFast()
8     func = None
9     for addr, f in cfg.kb.functions.items():
10         if f.name == "check_license":
11             func = f
12             break
13     if func is None:
14         print(f"[{label}] check_license bulunamadi!")
15         return None
16     blocks = list(func.blocks)
17     graph = func.transition_graph
18     in_degree = {}
19     for u, v in graph.edges():
20         in_degree[v] = in_degree.get(v, 0) + 1
21     n_blocks = len(blocks)

```

```

n_edges = graph.number_of_edges()
max_indeg_node = max(in_degree, key=in_degree.get) if in_degree else None
max_indeg = in_degree.get(max_indeg_node, 0)
print(f"\n=== (label) ===")
print(f"Fonksiyon adresi: {hex(func.addr)}")
print(f"Temel blok sayisi: {n_blocks}")
print(f"Kenar (edge) sayisi: {n_edges}")
if max_indeg_node is not None:
    print(f"En yuksek in-degree'ye sahip blok: {hex(max_indeg_node.addr)} if hasattr(max_indeg_node, 'addr') else max_indeg_node (in-degree={max_indeg})")
    is_dispatcher_candidate = max_indeg >= max(3, n_blocks * 0.4)
    print(f"Dispatcher adayi mi (heuristik: in-degree >= max(3, %40*blok)): {'EVET' if is_dispatcher_candidate else 'hayir'}")
return {"n_blocks": n_blocks, "n_edges": n_edges, "max_indeg": max_indeg}
orig = analyze("original_win.exe", "ORIGINAL (obfuscation yok)")
flat = analyze("flattened_win.exe", "CFF UYGULANMIS")
if orig and flat:
    print("\n=== KARSILASTIRMA (Cizelge 3.2 icin gercek veri) ===")
    print(f"{'Metrik':<30} {'Orjinal':<12} {'CFF':<12}")
    print(f"{'Temel blok sayisi':<30} {orig['n_blocks']:<12} {flat['n_blocks']:<12}")
    print(f"{'Kenar sayisi':<30} {orig['n_edges']:<12} {flat['n_edges']:<12}")
    print(f"{'Max in-degree':<30} {orig['max_indeg']:<12} {flat['max_indeg']:<12}")

```

5. Stage3_symbolic_explore.py ve stage3_compare.py

```

1 import angr
2 import claripy
3 import logging
4 logging.getLogger('angr').setLevel(logging.ERROR)
5 logging.getLogger('cle').setLevel(logging.ERROR)
6 proj = angr.Project("flattened_win.exe", auto_load_libs=False)
7 cfg = proj.analyses.CFGFast()
8 func = None
9 for addr, f in cfg.kb.functions.items():
10     if f.name == "check_license":
11         func = f
12         break
13 print(f"check_license adresi: {hex(func.addr)}")
14 code_sym = claripy.BVS('code', 32)
15 state = proj.factory.call_state(func.addr, code_sym, cc=proj.factory.cc())
16 simgr = proj.factory.simulation_manager(state)
17 simgr.explore(n=1000)
18 print(f"active={len(simgr.active)} deadended={len(simgr.deadended)} errored={len(simgr.errored)}")
19 # hata nedenini gorelim
20 for e in simgr.errored[:3]:
21     print("ERROR:", e.error)
22 def to_signed32(v):
23     return v - 2**32 if v >= 2**31 else v
24 print("\n=== Kurtarilan (code -> result) esleme ornekleri ===")
25 seen = set()
26 for st in simgr.deadended:
27     try:
28         ret_val_u = st.solver.eval(st.regs.rax, cast_to=int)
29         ret_val = to_signed32(ret_val_u & 0xFFFFFFFF)
30         code_v_u = st.solver.eval(code_sym, cast_to=int)
31         code_v = to_signed32(code_v_u)
32         key = (code_v, ret_val)
33         if key not in seen:
34             seen.add(key)
35             print(f"code={code_v:>6} -> result={ret_val}")
36     except Exception as ex:
37         print("eval error:", ex)

```

```

40 def classify(code, ret):
41     if code < 0: return "NEG (result=-1)"
42     if code == 0: return "ZERO (result=0)"
43     if (code * 2) == ret: return "MOD_TRUE (result=code*2)"
44     if (code + 100) == ret: return "MOD_FALSE (result=code+100)"
45     return "BILINMEYEN"
46
47 print("\n--- Orijinal binary'de kesfedilen dallar ---")
48 for c, r in map_o:
49     print(f"   code={c:>15}   result={r:<15}   sinif={classify(c,r)}")
50
51 print("\n--- CFF binary'de kesfedilen dallar ---")
52 for c, r in map_f:
53     print(f"   code={c:>15}   result={r:<15}   sinif={classify(c,r)}")
54
55 classes_o = sorted(set(classify(c,r) for c,r in map_o))
56 classes_f = sorted(set(classify(c,r) for c,r in map_f))
57 print(f"\nOrijinal'de bulunan davranis siniflari: {classes_o}")
58 print(f"CFF'de bulunan davranis siniflari:      {classes_f}")
59 print(f"Davranissal esdegerlik (ayni sinif kumesi mi?): {'EVET' if classes_o == classes_f else 'HAYIR'}")

```

```

def to_signed32(v):
    v &= 0xFFFFFFFF
    return v - 2**32 if v >= 2**31 else v

def symbolic_explore(path):
    proj = angr.Project(path, auto_load_libs=False)
    cfg = proj.analyses.CFGFast()
    func = None
    for addr, f in cfg.kb.functions.items():
        if f.name == "check_license":
            func = f
            break
    code_sym = claripy.BVS('code', 32)
    state = proj.factory.call_state(func.addr, code_sym, cc=proj.factory.cc())
    simgr = proj.factory.simulation_manager(state)
    simgr.explore(n=1000)

    mappings = []
    for st in simgr.deadended:
        try:
            ret_u = st.solver.eval(st.regs.rax, cast_to=int)
            code_u = st.solver.eval(code_sym, cast_to=int)
            mappings.append((to_signed32(code_u), to_signed32(ret_u)))
        except Exception:
            pass
    return len(list(func.blocks)), len(simgr.deadended), mappings

n_blocks_o, n_paths_o, map_o = symbolic_explore("original_win.exe")
n_blocks_f, n_paths_f, map_f = symbolic_explore("flattened_win.exe")

print("=== KARSILASTIRMA: Sembolik Yol Kesfi Sonuclari ===\n")
print(f"{' ':30}{'Orijinal':<15}{'CFF':<15}")
print(f"{'Temel blok sayisi':30}{n_blocks_o:<15}{n_blocks_f:<15}")
print(f"{'Kesfedilen distinct yol sayisi':30}{n_paths_o:<15}{n_paths_f:<15}")

```

ÖZGEÇMİŞ

KİŞİSEL BİLGİLER

Adı Soyadı : Senanur Özbağ
Doğum Tarihi ve Yeri : 01/01/2003
Yabancı Dili : İngilizce C2
E-posta : senanurozbag16@gmail.com

ÖĞRENİM DURUMU

Derece	Alan	Okul/Üniversite	Mezuniyet Yılı
Lisans	Bilgisayar Müh.	Kastamonu Üniversitesi	2026
Lise	SAYISAL	Özel Eğitim Bilimleri Koleji	2021

